



你每天用的 App，背後長什麼樣？



線上點餐 App

從 10 個朋友開始，小小的。
你負責開發，我來解釋背後發生什麼。
準備好了嗎？系統等等就要爆了。





App 就像一家餐廳

客人點餐 → 服務生傳單 → 廚房做菜

冰箱存料、菜單放桌上、帳單算完收錢

每個角色各司其職，缺一不可



三個你要記住的角色

Client / 客人端

你的手機、瀏覽器，負責送出請求

Server / 伺服器

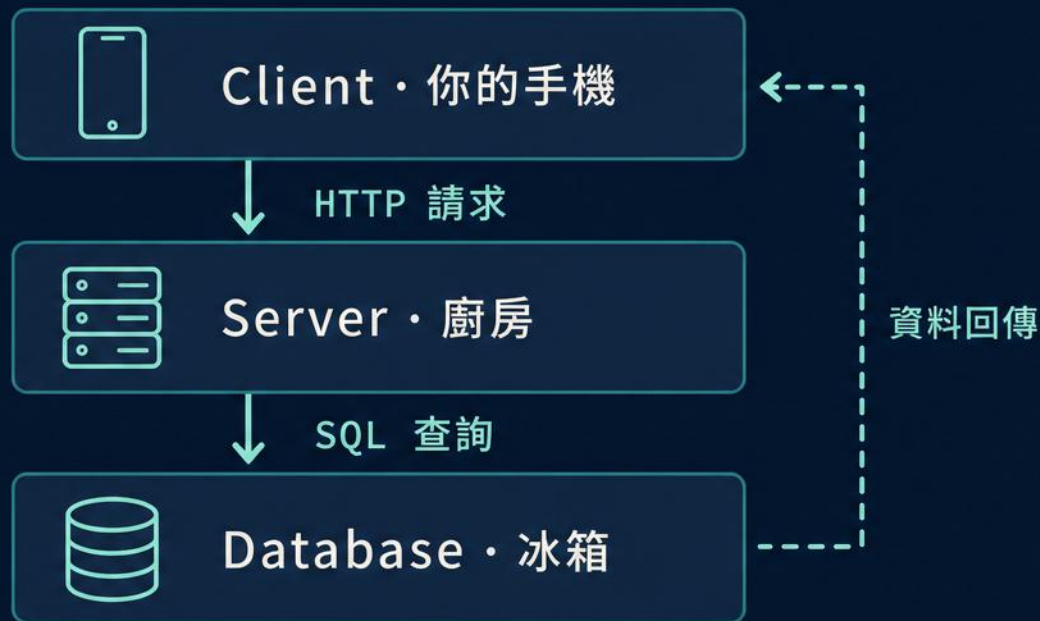
廚房，接收請求並處理邏輯

Database / 資料庫

冰箱，負責存放與讀取所有資料



點一份蛋餅，發生什麼？



- ① 手機送出「我要蛋餅」的請求
- ② Server 收到，去 DB 查有沒有庫存
- ③ DB 回資料，Server 整理後送回手機



這就是最簡單的系統



一支手機、一台 Server、一個 DB

三個方塊，串起整個服務

之後每個決定，問這四題

C

一致性

Consistency

所有人看到的資料一樣嗎？

A

可用性

Availability

系統掛掉了還能用嗎？

L

延遲

Latency

多快回應用戶的請求？

Cost

成本

Cost

要燒多少錢才能撐住？

沒有最好的答案，只有取捨。



一台機器就夠了…嗎？

人變多，一台 Server 開始喘不過氣。

下一章，系統就要爆了。

畫給我看

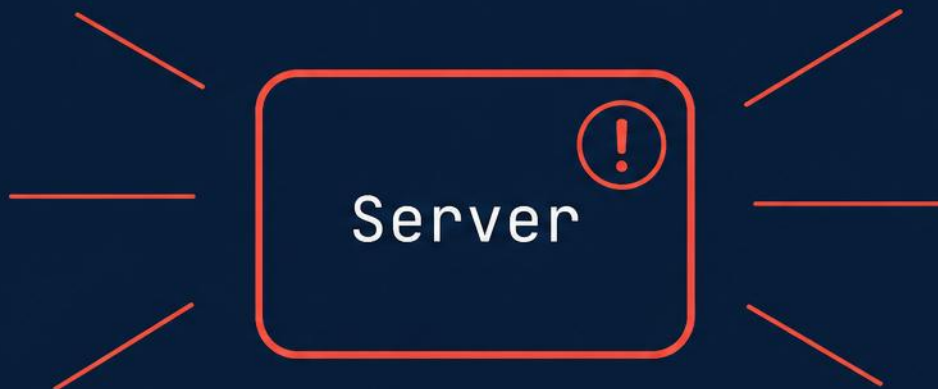
闖上這頁，默畫架構圖 v1。三個方塊，一條箭頭串起來。

下章揭曉：人變多，Load Balancer 登場



PAIN POINT

服務 1 萬用戶



中午 12 點，系統整個掛了

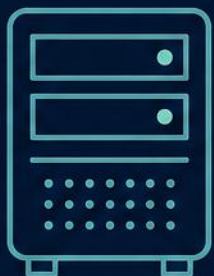
訂單全湧進來，一台 Server 扛不住。

用戶按送出——轉圈圈、無回應、崩潰。

你的 App 在最忙的時候讓大家失望。



問題很簡單：只有一台



1 台 Server



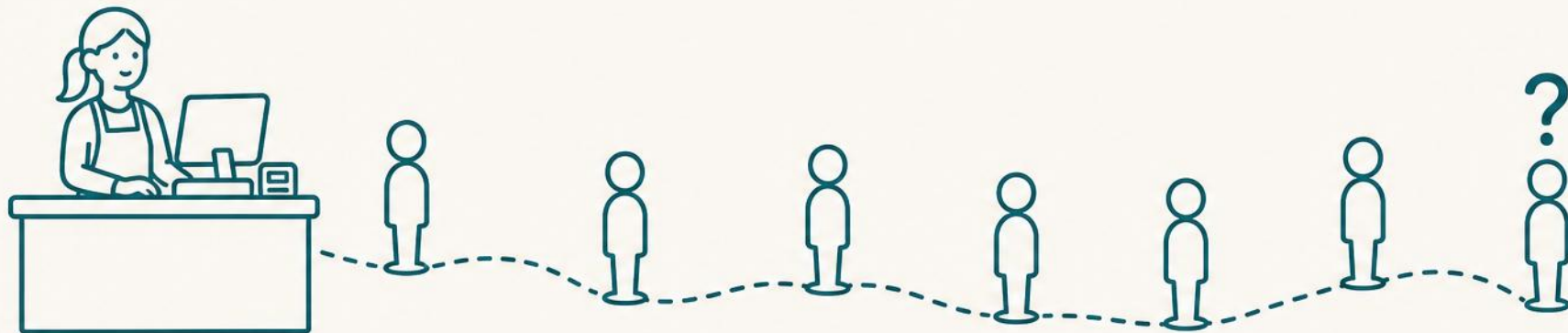
10,000 請求

一台 Server，記憶體有上限、CPU 有上限。

同時湧入一萬個請求，資源瞬間耗盡。

換一台更大的機器？只是延後問題。





只開一個收銀台，會怎樣？

中午人潮湧入，一個收銀員拚命結帳。

後面的客人等到天荒地老，氣到走人。

解法不是換更快的收銀員——是多開幾台。



兩個詞彙，解決「人太多」

Load Balancer / 負載平衡器

把請求分配給多台 Server，不讓一台撐死

Horizontal Scaling / 水平擴展

加機器而非換更大的機器，可以無限延伸



Server 不能自己記東西

Stateless 無狀態

每個請求帶齊所有資訊，Server 不記任何狀態



你登入後，下一個請求可能被不同 Server 處理。

如果 Server 自己記登入狀態，就會出問題。

解法：把狀態外移到 DB 或 Session Store。



架構長出新方塊了



架構圖 v2

Load Balancer 站在最前面，分流請求。
後面多台 Server 一起扛，不再一台孤軍。



擴展性變好，代價是什麼？

Uber、蝦皮尖峰都靠負載平衡分流。

但系統更複雜，Session 要外移，成本升高。

C

Session 外移，
一致性設計複雜度升高

A

↑ 一台掛掉，其他繼續服務

L

多一層轉發，延遲微增

Cost

↑ 多台機器，費用倍增

沒有最好的答案，只有取捨。



闖上這頁，默畫架構圖 v2

手機 → Load Balancer → 兩台 Server → DB

哪個方塊是新的？邊框是什麼顏色？

下章：DB 被打爆了，怎麼辦？

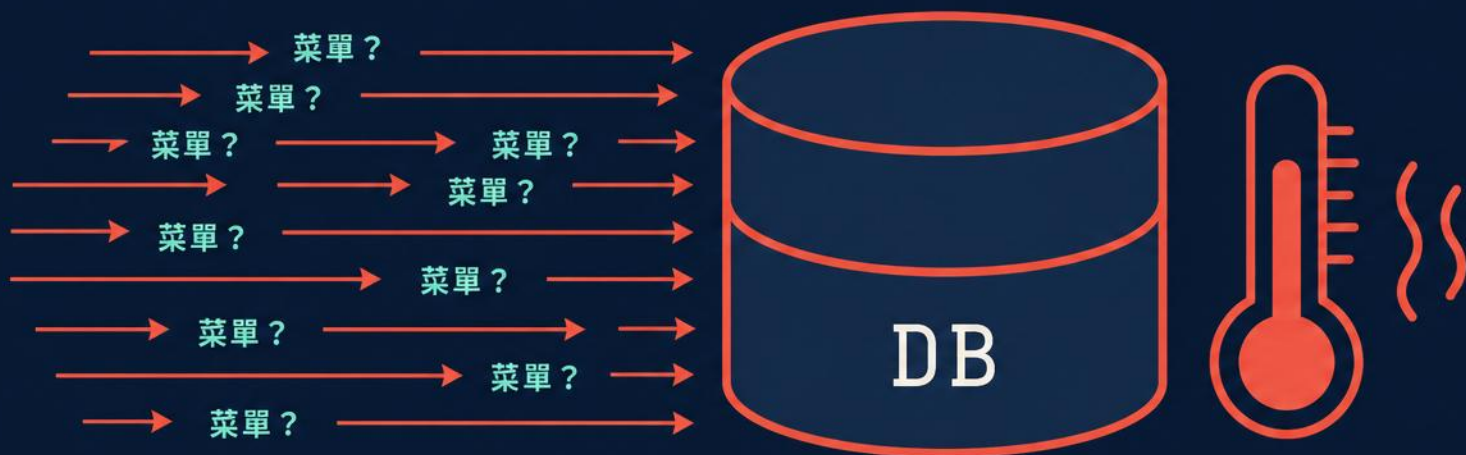
畫給我看

闖上這頁，默畫 v2。Load Balancer 放在哪？幾台 Server？

下章揭曉：大家瘋狂刷菜單，DB 快打爆了



DB 快撐不住



每次打開 App，都去 DB 查一次菜單。

十萬個人同時查，DB 回應越來越慢。

菜單根本沒變——卻一直重複查一樣的資料。



同一份菜單，查了十萬次

菜單更新次數



1

← VS →

DB 查詢次數



100,000

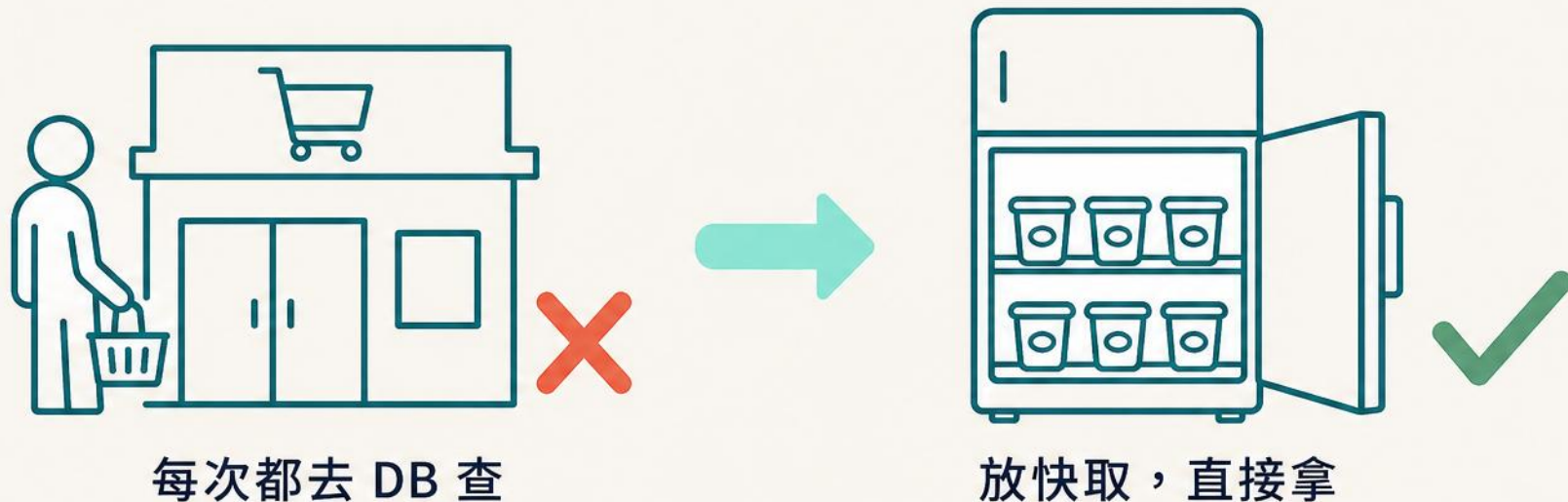
菜單一天只改一次，但十萬個人各查一次。

DB 被一樣的問題問到崩潰。

這不是流量問題，是設計問題。



常吃的東西，放冰箱就好



每次想吃優格，都跑一趟超市——很蠢。

聰明的做法：買一批，放家裡冰箱，要吃就拿。

「冰箱」就是快取——把熱資料放在手邊。



快取三個詞，先背起來

Cache / 快取

把熱資料存在記憶體，下次直接拿，不去 DB

Cache Hit / 快取命中

資料在快取裡找到了，速度快，不打 DB

Cache Miss / 快取未命中

快取沒有，只好去 DB 查，然後存回快取



快幾萬倍——不誇張

記憶體



~100 ns

跨網路 DB



~10 ms

差距：× 100,000 倍

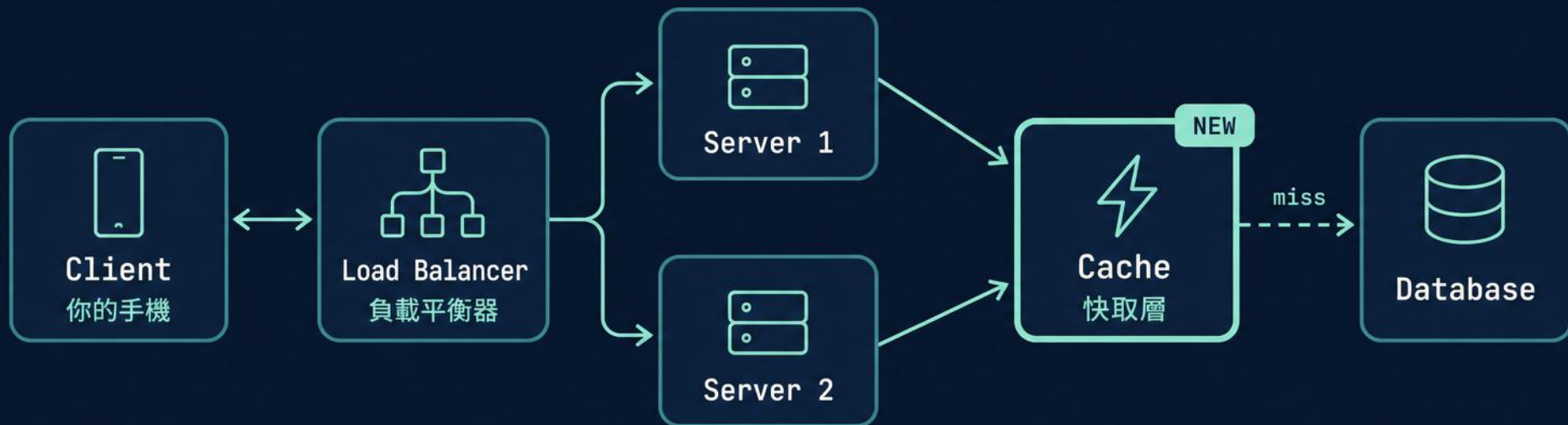
記憶體讀取：約 100 奈秒 (ns)。

跨網路讀 DB：約 1-10 毫秒 (ms)。

快取命中比去 DB 快了一萬倍。



架構中間多了一層快取



架構圖 v3

Server 先問快取，有就直接回，不打 DB。

快取沒有，才去 DB 查，然後存進快取。



速度快了，但菜單可能是舊的

幾乎所有大站都靠快取扛讀取流量。

代價：快取沒更新時，客人看到舊菜單。

C

↓ 快取未更新時，
資料可能是舊的

A

DB 壓力降低，
整體更穩定

L

↓ 大幅降低，
命中時快幾萬倍

Cost

記憶體比 DB 貴，
但減少 DB 擴展費用

沒有最好的答案，只有取捨。



闔上這頁，默畫架構圖 v3

v2 基礎上，在 Server 和 DB 之間加一個 Cache。

哪條箭頭是 hit？哪條是 miss？

下章：訂單爆量，一個 DB 放不下了。

畫給我看

闔上這頁，默畫 v3。Cache 放在哪？兩條箭頭分別代表什麼？

下章揭曉：訂單爆量，一個 DB 再也不夠用了



PAIN POINT

服務 100 萬用戶

一個 DB 不夠



一百萬個訂單，DB 寫入速度越來越慢。

容量快滿了，每次查詢都要等更久。

一台 DB 壞掉，所有訂單全部消失。



一個 DB，同時面對兩個危機

危機一

讀寫太慢

DB 回應越來越慢，撐不住請求量

+

危機二

容量不夠

一台 DB 硬碟有上限，塞不下更多資料

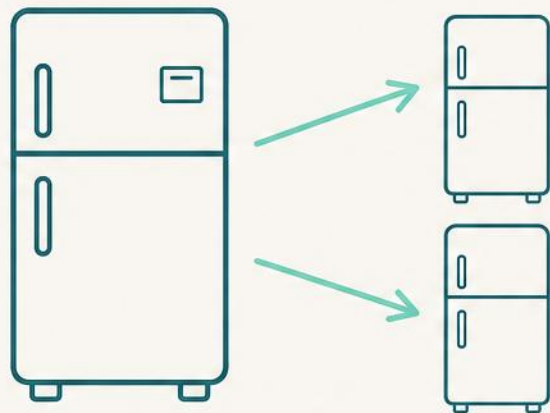
危機一：讀寫請求太多，DB 跑不動。

危機二：容量快滿，資料快放不下。

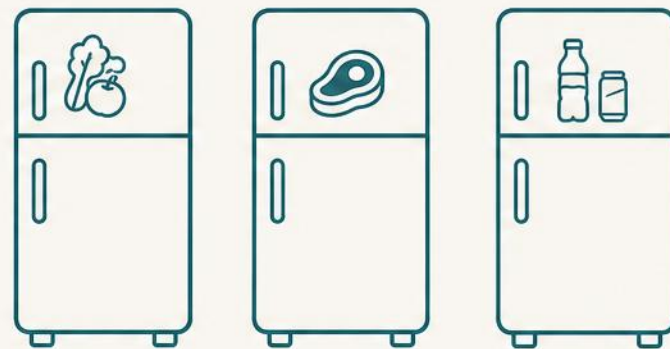
這兩個問題需要不同的解法。



一台冰箱不夠，怎麼辦？



複本：多備幾台，讀取分流



分片：按類型切開，各放各的

常用的食物多備幾台冰箱——壞了不怕，備份有。

東西太多放不下，就依種類分不同冰箱存。

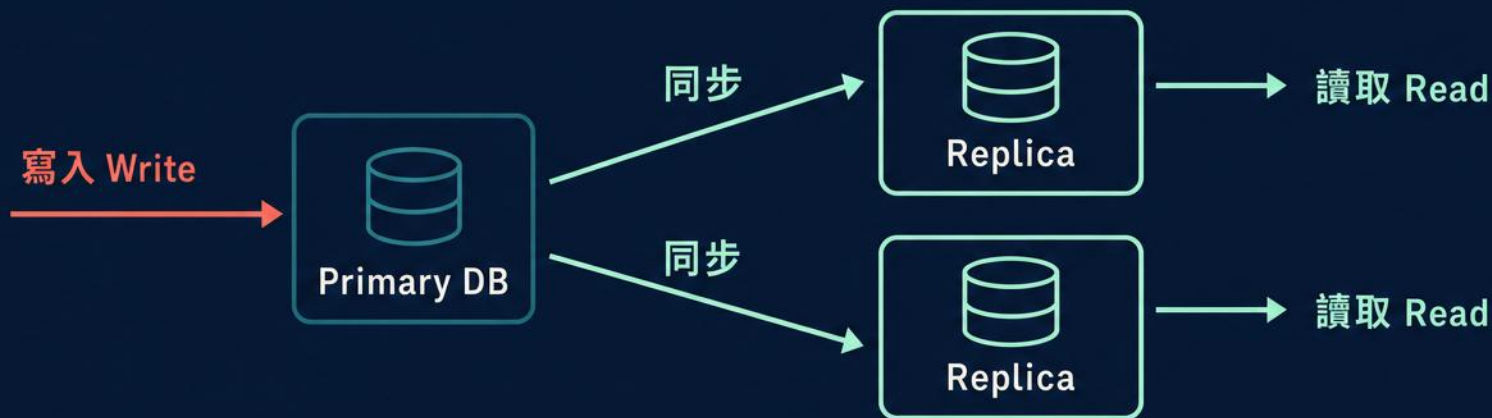
複本解決「讀太慢」，分片解決「放不下」。



讀太慢？加複本來分擔

Read Replica / 讀取複本

同步主 DB 資料的副本，專門分擔讀取請求



主 DB (Primary) 負責所有寫入。

複本 (Replica) 只負責讀取，同步主 DB 的資料。

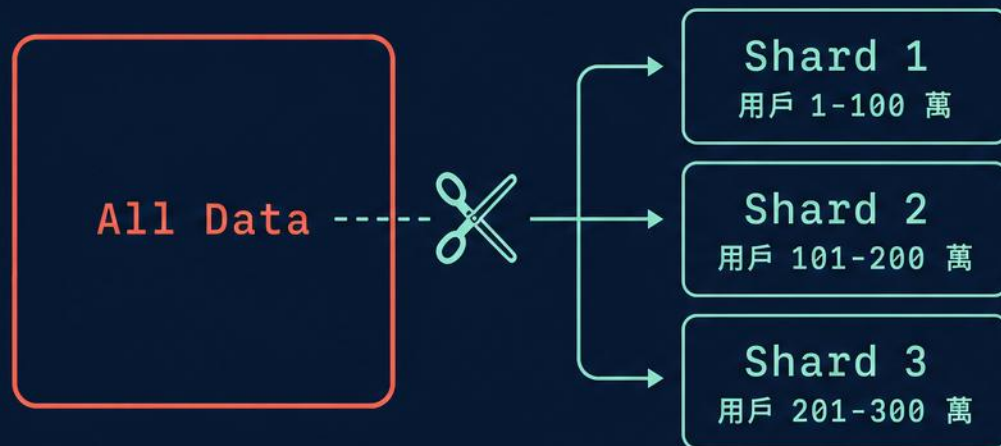
讀寫分離，讓兩邊都不會太忙。



放不下？把資料切開存

Sharding / 分片

把資料依規則切分，存到多台 DB，解決容量問題



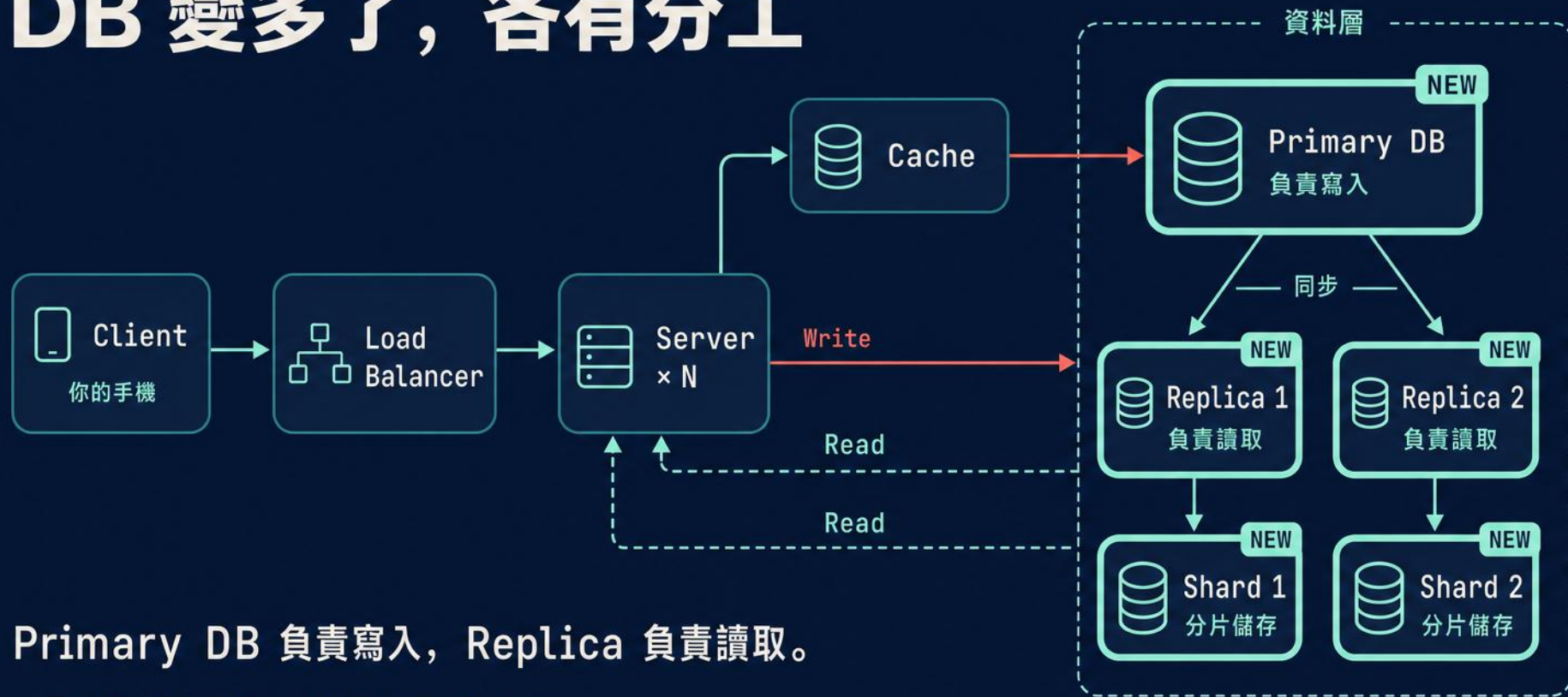
把資料依規則切成多份，各存到不同 DB。

例如：用戶 1-100 萬存 DB1，101-200 萬存 DB2。

每台 DB 只存一部分，容量問題解決。



DB 變多了，各有分工



Primary DB 負責寫入，Replica 負責讀取。

資料依規則切片，分存到多台 DB。

大電商的訂單系統都長這樣

大型電商訂單庫都分片加複本，撐住百億資料。

但複本可能短暫不同步，跨片查詢變複雜。

C

↓ 複本同步有延遲，
可能短暫讀到舊資料

A

↑ 一台壞掉，
複本仍可服務

L

讀取分流，延遲改善；
跨片查詢延遲升高

Cost

↑ 多台 DB，成本倍增，
維護複雜度也升高

沒有最好的答案，只有取捨。



闖上這頁，默畫架構圖 v4

v3 基礎上，DB 區域長出 Primary + Replica + 分片。

哪條箭頭是寫入？哪條是讀取同步？

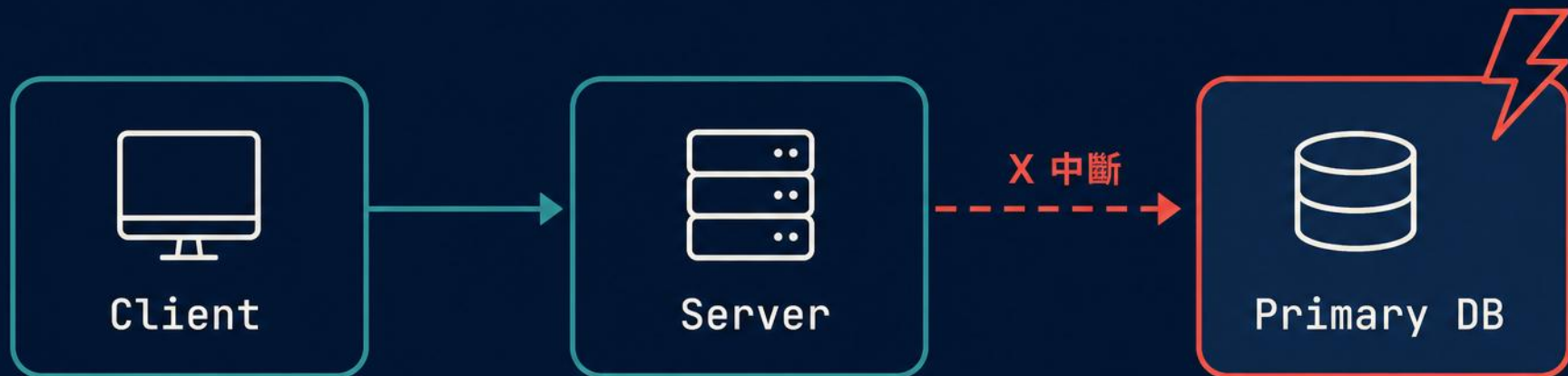
畫給我看

闖上這頁，默畫 v4。Primary 和 Replica 各負責什麼？分片怎麼畫？

下章揭曉：機器會壞，系統要能撐住



機器會掛，網路會斷



Primary DB 突然離線，服務全面中斷。

Server 重啟中，請求全部逾時等待。

一個節點壞，整個系統就跟著倒。



網路一斷，訂單重複了

送出 #1

訂單 A

\$320 元

× 2

送出 #2 (重複)

訂單 A

\$320 元

扣款兩次

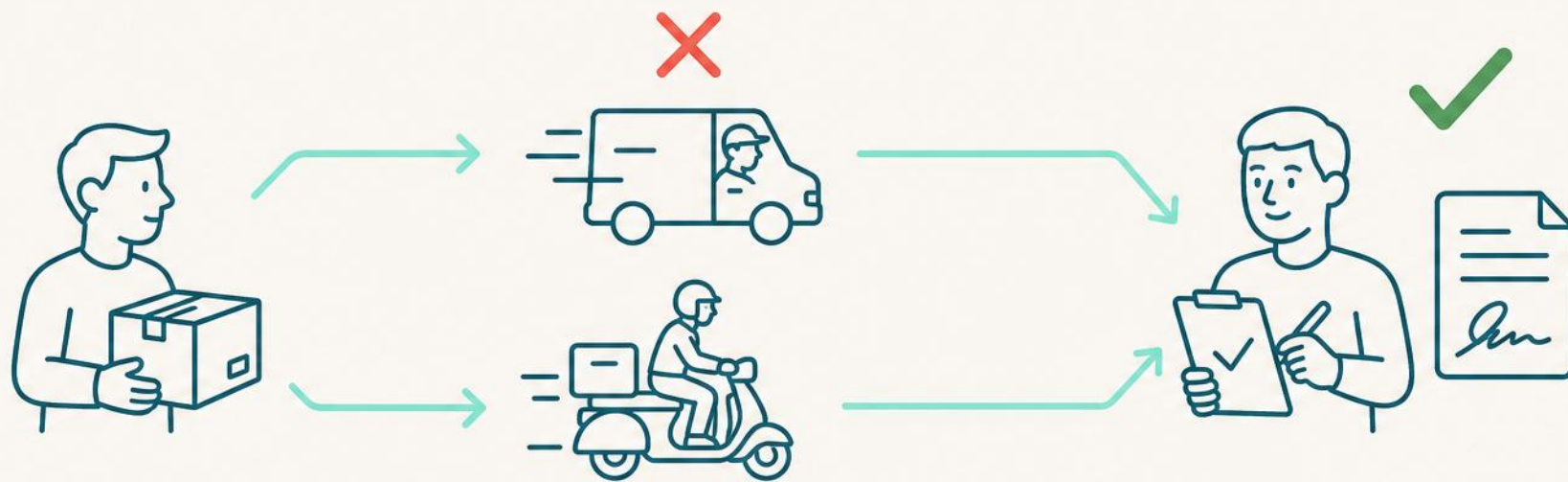
客人送出訂單，網路卡住沒收到回應。

客人再按一次，Server 收到兩筆訂單。

帳戶重複扣款，客服電話打爆。



寄重要包裹要簽收，也要備份



簽收確認：包裹送到才算完成，沒收到重寄。

備份路線：主要快遞掛了，換另一條路送。

不能寄丟，也不能因重寄而送兩份。



掛了就切，切了再試

Failover / 容錯備援

主機掛了，自動切到備援，服務不中斷

Retry / 重試

請求失敗後自動重新送出，等待系統恢復

Failover：主機掛了，自動切到備援節點。

健康檢查持續偵測各節點是否存活。

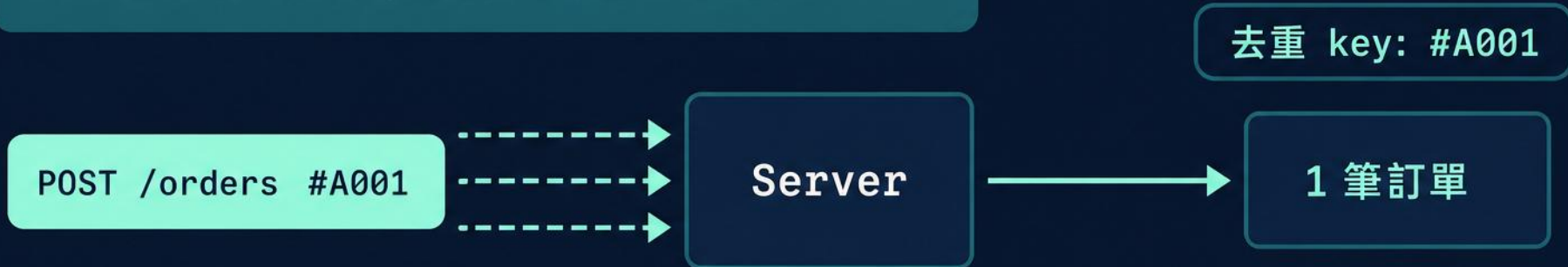
Retry：失敗了自動重送，但要小心重複。



做幾次都一樣

Idempotency / 冪等

同一操作執行多次，效果與執行一次相同



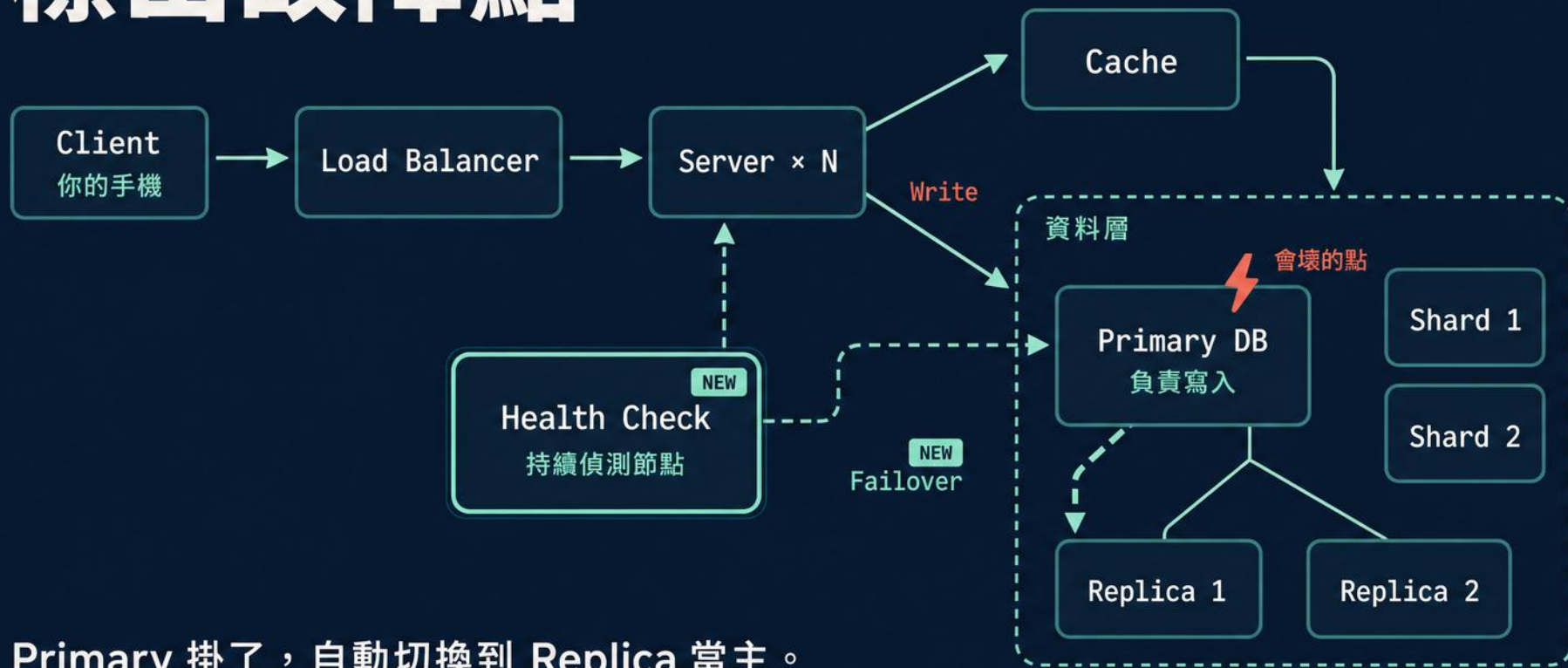
送出「訂單 #A001」，不管送幾次只建一筆。

靠唯一訂單編號去重，防止重複扣款。

Retry 安全的前提，就是操作必須冪等。



標出故障點



Primary 掛了，自動切換到 Replica 當主。

健康檢查隨時偵測，發現掛掉立即切換。

金流系統靠冪等防重複扣款

Stripe 等金流每筆請求帶唯一 key，去重防重複。

付款要「強一致」；看菜單可以「最終一致」。

付款 → 強一致 C 優先

看菜單 → 最終一致 A/L 優先

C

↑ 冪等保障付款不重複，
一致性提升

A

↑ Failover 備援讓系統
掛了仍可切換服務

L

健康檢查偵測+切換有秒級
延遲，可接受

Cost

↑ 備援節點+健康檢查服務，
成本增加

C/A/L/Cost：每個決定都是四維取捨，沒有免費的保障。



闖上這頁，默畫架構圖 v5

v4 基礎上，加上健康檢查方塊與備援路徑。

哪個節點標了「會壞的點」？Failover 往哪切？

下章：圖片和影片，DB 要怎麼存？

畫給我看

默畫 v5。在 Primary DB 旁邊加一個會壞的點標示，再加一條 Failover 路徑。

下章揭曉：照片和影片，塞爆 DB 怎麼辦？



PAIN POINT

服務 300 萬用戶

食物照片塞爆了資料庫



95% FULL

每家店上傳十張菜單照，三百萬用戶每人存幾張。

DB 存不下，磁碟空間快滿，備份極慢。

圖片載入要等五秒，用戶早就離開了。



圖片太慢，用戶等不了



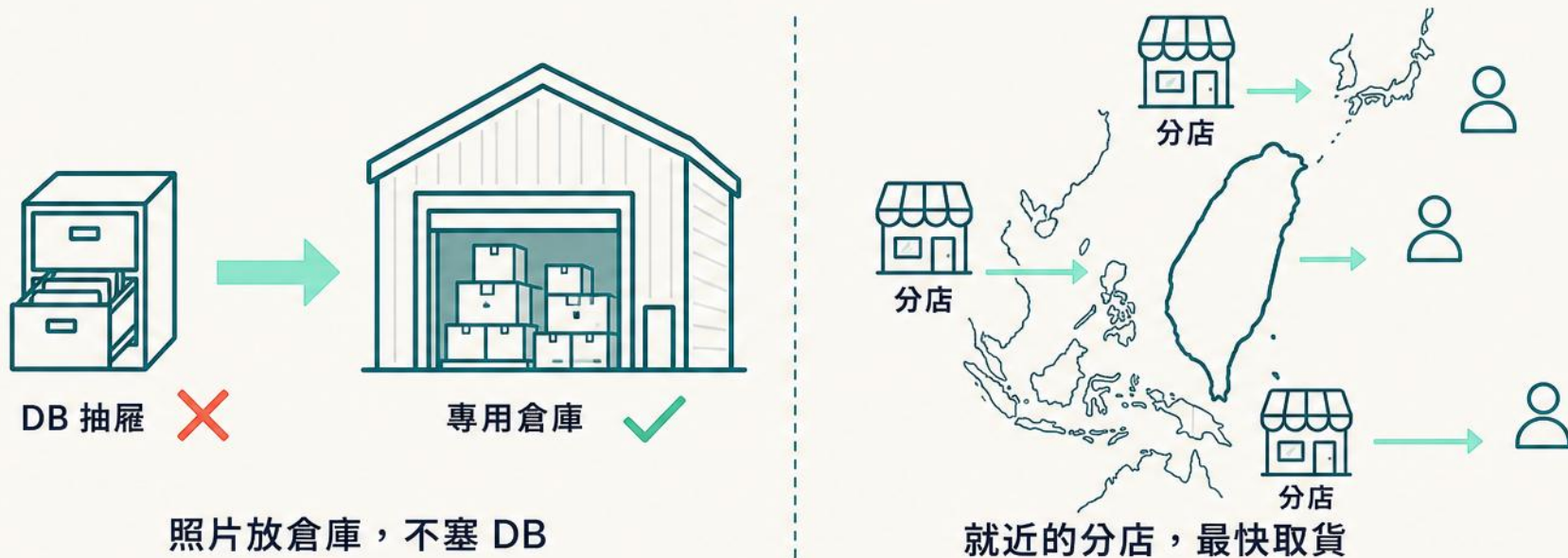
台灣用戶連到美國的伺服器，每張圖要等三秒。

尖峰時段大量請求，Server 傳圖頻寬被佔滿。

用戶體驗差，直接關掉換競品。



照片放倉庫，各地開分店就近取



抽屜（DB）放文件；大量照片放專用倉庫。
全國連鎖店在各城市設分店，就近取貨最快。
Blob Storage 是倉庫，CDN 是就近的分店。



大檔案有專用倉庫

Blob Storage / 物件儲存

專門存放大型二進位檔案，不放進關聯式資料庫



圖片、影片、PDF，統一存進物件儲存。

DB 只存檔案位置（URL），不存原始檔案。

儲存成本低，容量近乎無限，方便備份。



就近快取檔案

CDN / 內容傳遞網路

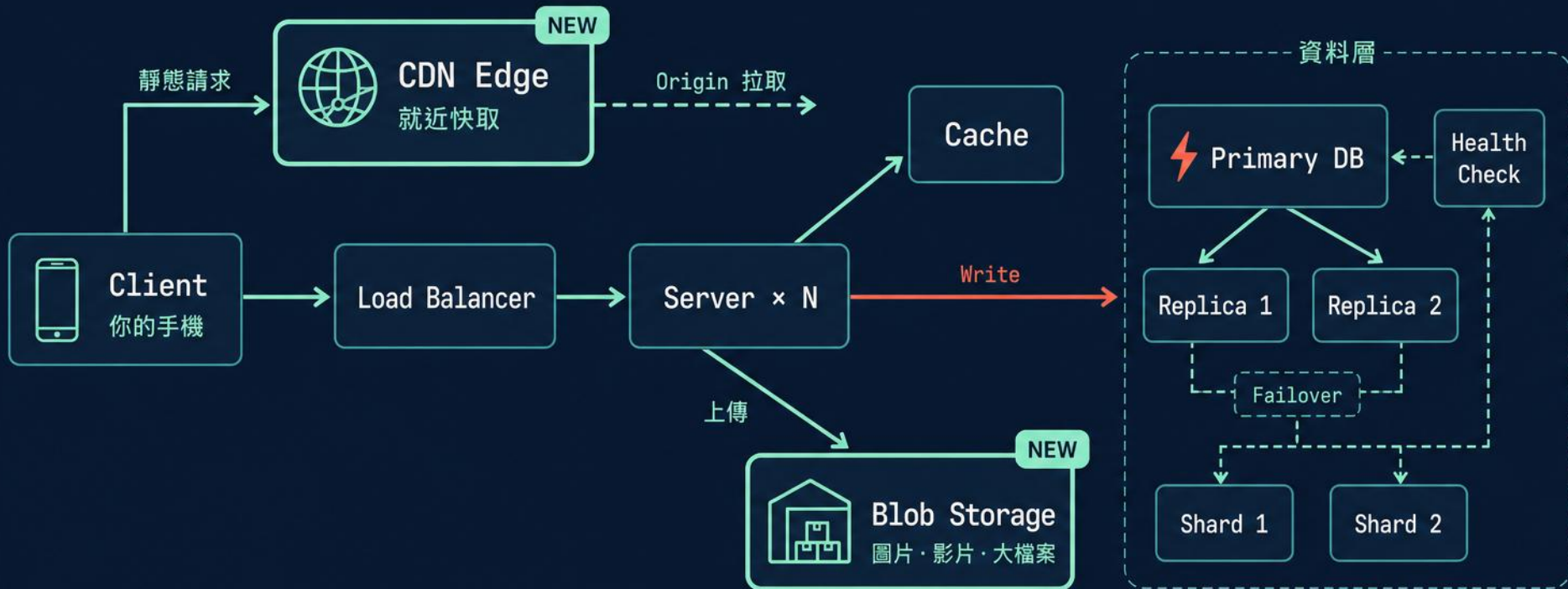
把靜態資源快取到全球各地節點，就近服務用戶



CDN 在全球各地佈署快取節點 (Edge) 。
用戶請求圖片，從最近的節點回傳，毫秒級。
Origin Server 壓力大減，只需首次傳輸。



CDN 與倉庫



圖片和影片存進 Blob Storage，DB 只存 URL。

CDN 節點在用戶附近，圖片秒開。

CDN 撐起影音

YouTube 每分鐘上傳 500 小時影片，全靠物件儲存。

Netflix 在全球 4000+ 節點快取，才能一秒開片。

C

CDN 快取可能短暫落後，
更新要等快取過期

A

↑ CDN 節點分散，
Origin 掛了 Edge 仍可服務

L

↑ 就近快取，圖片和影片
載入速度大幅提升

Cost

↑ CDN 流量費用 +
Blob Storage 費用，
但 DB 成本降低

沒有最好的答案，只有取捨。



闖上這頁，默畫架構圖 v6

v5 基礎上，Client 前加 CDN，Server 旁加 Blob Storage。

靜態資源走哪條路？動態 API 走哪條路？

下章：訂單做好了沒？一直刷新很煩。

畫給我看

默畫 v6。在 Client 前加 CDN Edge，在 Server 旁邊加 Blob Storage。兩條路徑要畫清楚。

下章揭曉：請求太慢？用佇列讓客人先去坐著等。



服務 500 萬用戶

餐還沒好，客人卻在狂刷新



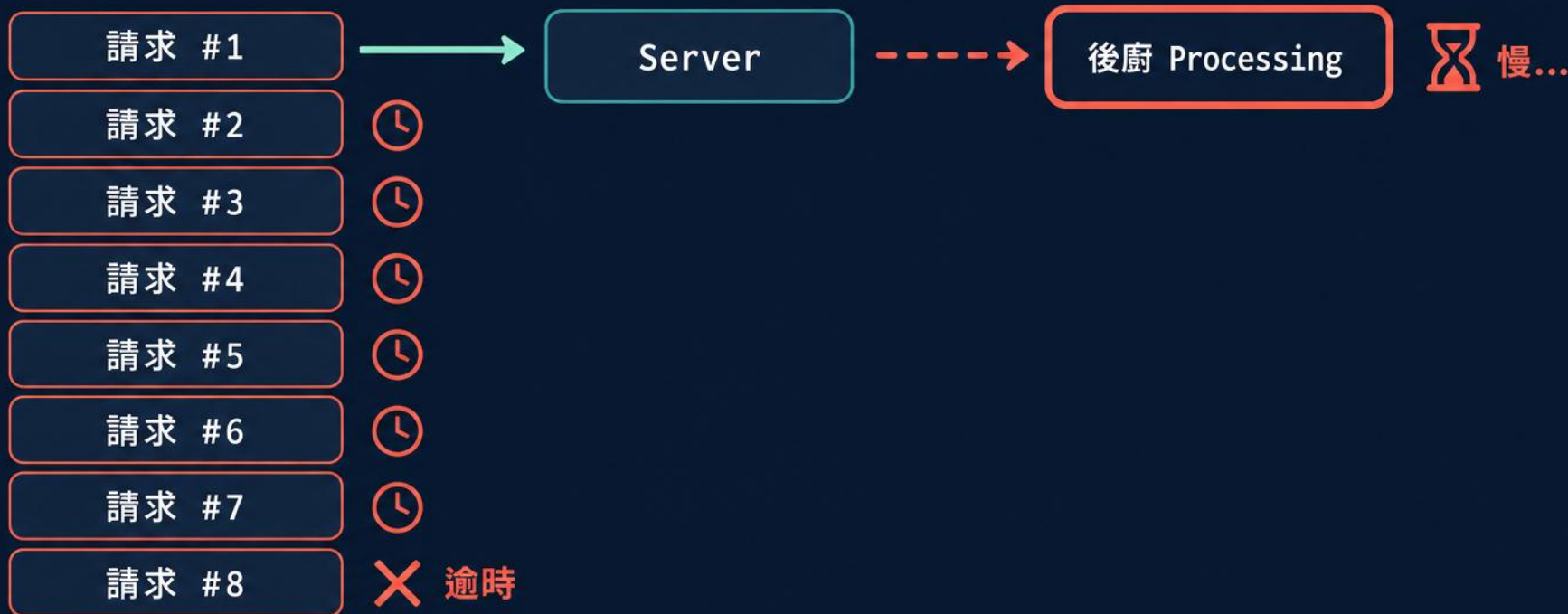
下單後 Server 同步等待後廚，請求卡住不動。

五百萬用戶每人每秒刷新，Server 被佔滿。

後廚慢一點，前台全部塞車。



一個慢請求，卡住所有後面的



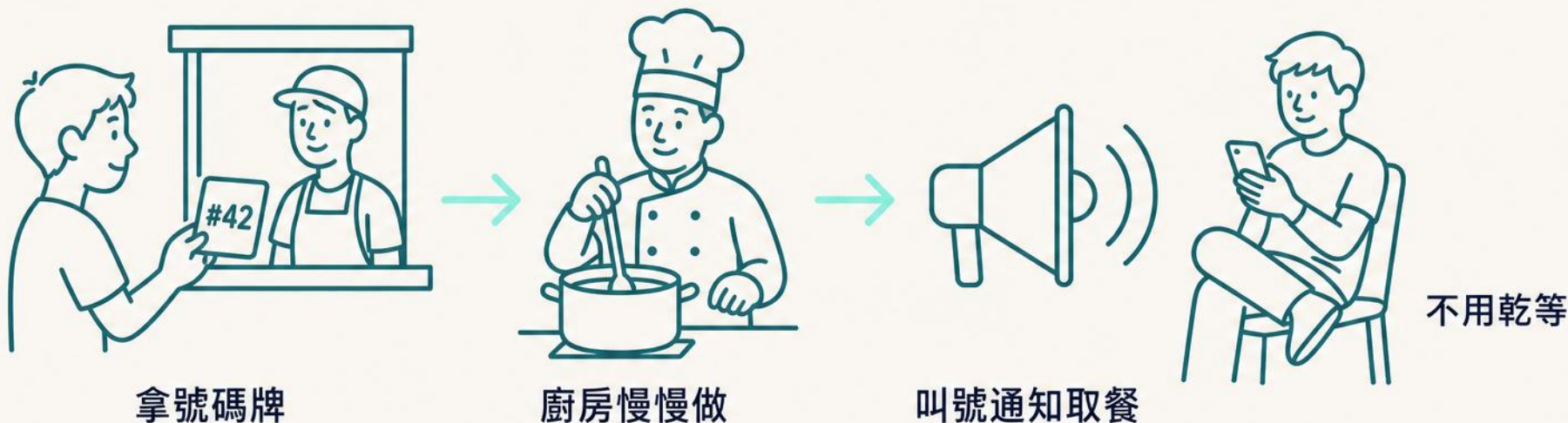
同步處理：Server 等後廚做完才回應。

尖峰時段一百個請求同時進來，全部排隊。

最後進來的用戶，等到逾時直接報錯。



號碼牌：拿了就去坐，叫號再來



點完餐拿號碼牌，去坐著，廚房慢慢做。

好了叫號通知，客人再來取餐。

窗口不堵車，客人也不用乾等。



先排隊再處理

Message Queue / 訊息佇列

請求先排隊等候，Worker 依序取出處理，解耦生產與消費

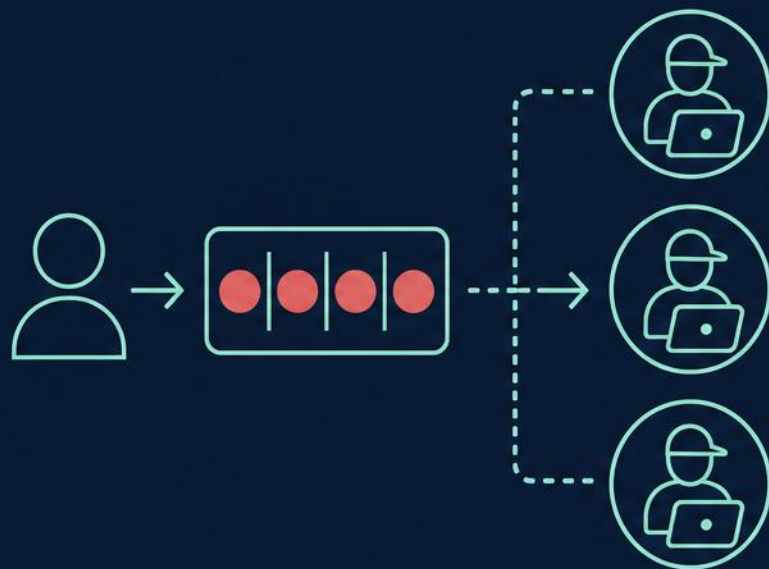
Worker / 背景工作

從 Queue 取出任務，在背景非同步執行，不占用 Server 主線程

Queue：請求進來先排進佇列，立刻回應客人。

Worker：背景慢慢從 Queue 取出任務處理。

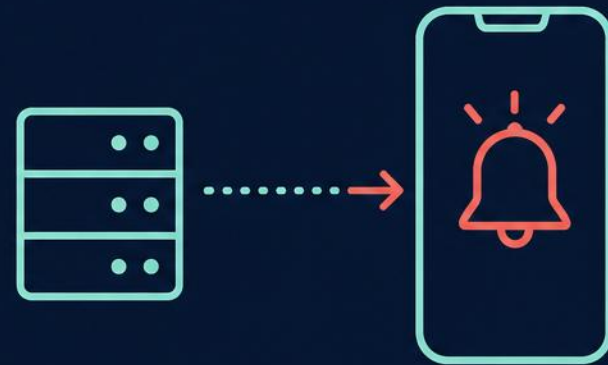
Server 解放，尖峰流量不再塞車。



怎麼通知客人

Push Notification / 推播通知

Server 主動推送訊息給 Client，Client 不需輪詢



Long Polling / 長輪詢

Client 送出請求，Server 保持連線直到有消息才回應



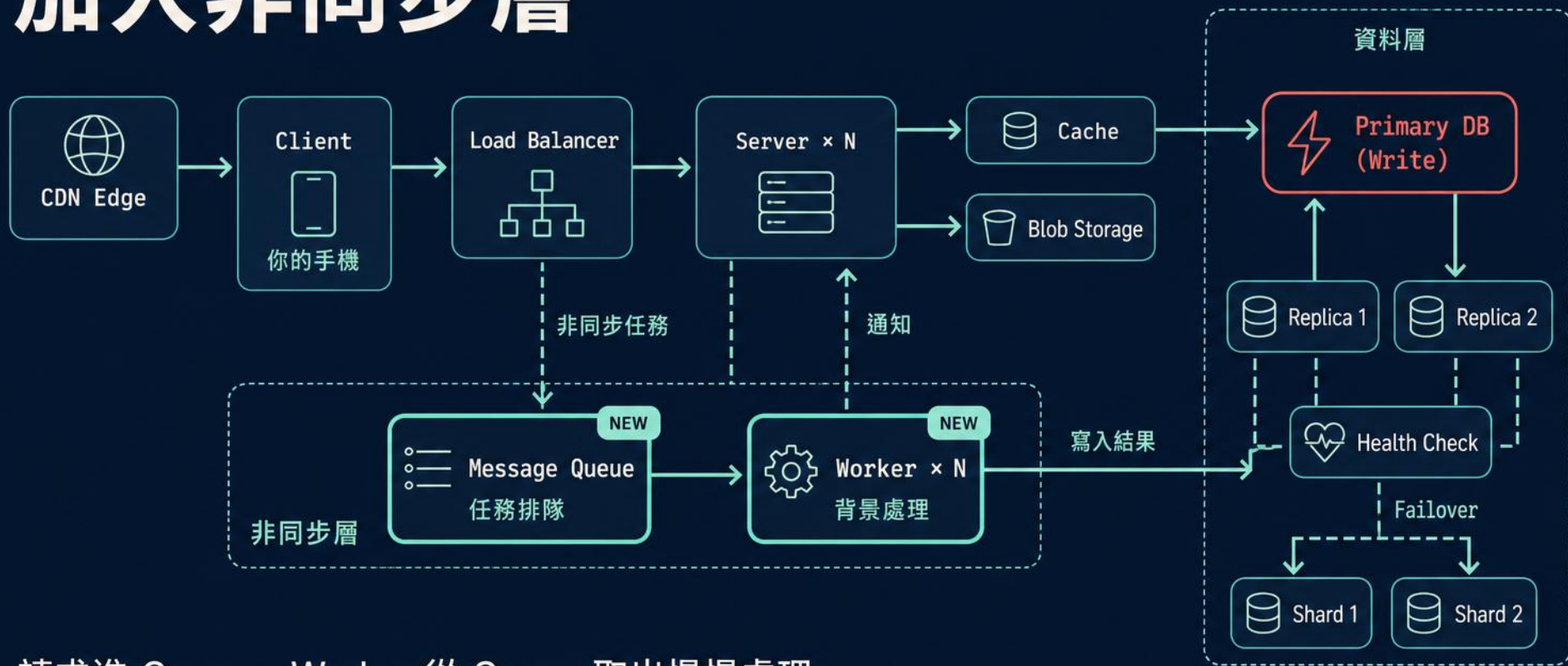
推播通知：Server 主動推送「你的餐好了」。

長輪詢：Client 問一次，Server 等到有消息才回。

兩種方式讓客人不用自己一直刷新。



加入非同步層



請求進 Queue，Worker 從 Queue 取出慢慢處理。

Server 只管接收，不再被慢任務拖住。



佇列加推播

Uber Eats 和 Foodpanda 用 Queue 處理訂單狀態。

解耦讓系統能扛尖峰；代價是結果不是即時的。

C

↓ 結果是最終一致，
使用者不能即時看到處理完成

A

↑ 解耦：後端慢或掛，
Queue 仍保住請求不遺失

L

Server 回應快，
但處理完成有時間差，
非零延遲

Cost

↑ Queue 服務 + Worker Pool，
增加架構複雜度與費用

沒有最好的答案，只有取捨。



闔上這頁，默畫架構圖 v7

v6 基礎上，Server 下方加一條箭頭進 Queue，Queue 接 Worker Pool。

Worker 做完後，結果往哪走？通知怎麼回？

恭喜：你已能默畫現代後端完整架構。

畫給我看

默畫 v7。在 Server 下方加 Message Queue，
Queue 右邊加 Worker Pool，再畫一條通知回去的箭頭。

v1 → v7，你已經走完了一個系統從零到現代的完整演化。



搜尋「附近蛋餅」，慢到逾時



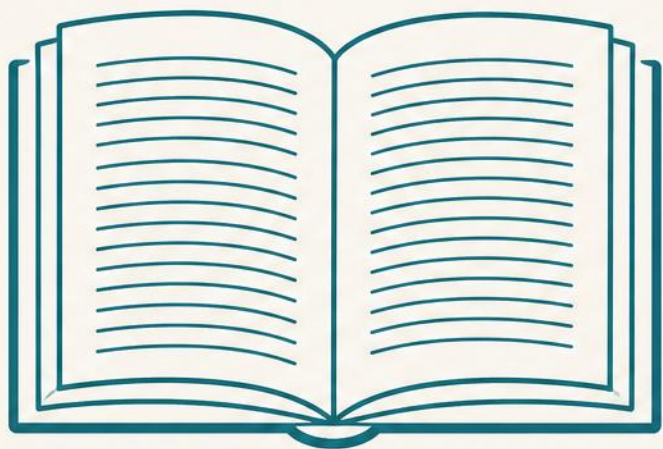
用 DB 的 LIKE 查詢，全表擰一遍才有結果。

五百萬筆菜單，一次搜尋讓 DB 喘不過氣。

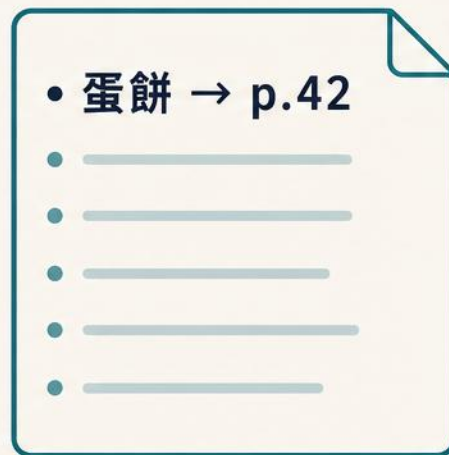
查不準、又慢，搜尋功能根本沒人用。



用索引找書



一頁一頁翻？太慢了



提前整理好，查詢秒回

想找「蛋餅」，不會從第一頁翻到最後一頁。

翻到書末索引，「蛋餅 → 第 42 頁」，直接跳過去。

索引是提前建好的，查的時候秒回。



預先建表，查詢秒回

Search Engine / 搜尋引擎

專為全文搜尋設計，能排序相關性，比 DB LIKE 快得多

Inverted Index / 反向索引

預先記錄「字 → 出現的文件列表」，查詢時直接跳到結果

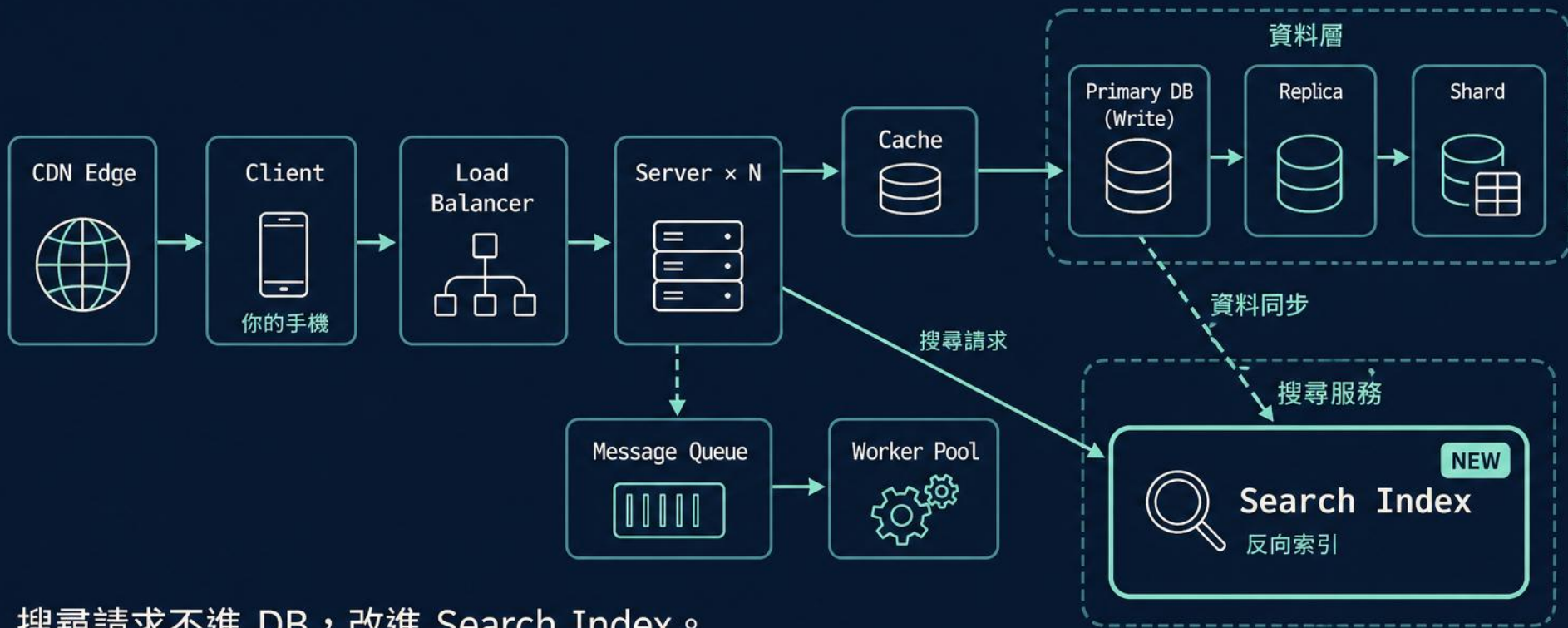
不等查詢時掃描，提前建好「字 → 文件」的對照表。

查「蛋餅」，秒找出所有含這個詞的菜單。

還能依相關性排序，最準的結果排最前。



搜尋旁掛



搜尋請求不進 DB，改進 Search Index。

資料寫入 DB 後，非同步同步給搜尋索引。



電商搜尋都是獨立搜尋引擎

蝦皮、Momo 等電商搜尋走 Elasticsearch 之類的引擎。

快又準；代價是多一套系統要維護。

C

↓ DB 寫入後索引非同步更新，
搜尋結果可能短暫落後

A

↑ 搜尋服務獨立，
搜尋掛了不影響主系統下單

L

↑ 反向索引查詢遠快於
DB LIKE 全表掃描

Cost

↑ 獨立搜尋服務的運算、
儲存費用，資料同步工程成本

沒有最好的答案，只有取捨。



默畫搜尋架構

v7 基礎上，右側旁掛一個 Search Index 方塊。

DB 畫一條虛線箭頭指向 Search Index (資料同步)。

恭喜完成選配章，你的架構圖升級為 v7+。

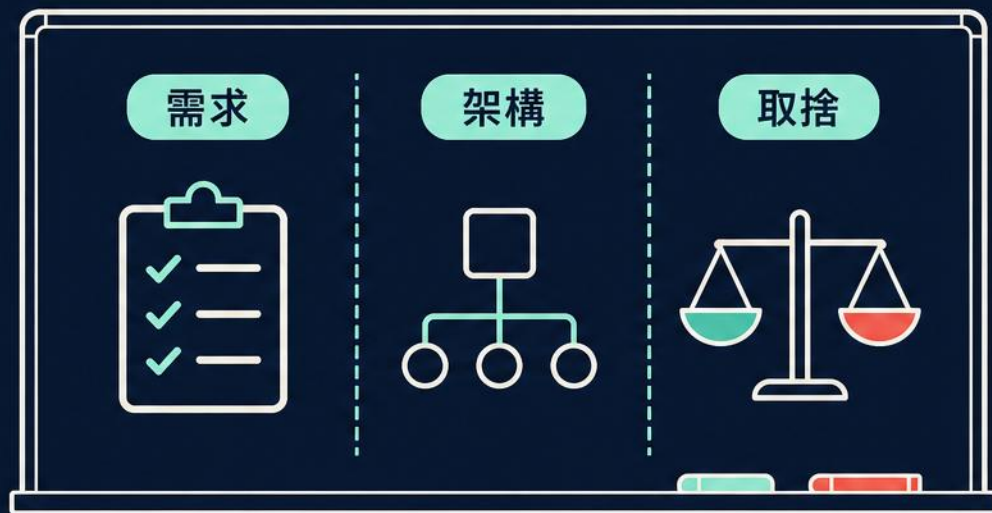
畫給我看

默畫 v7+。在 v7 右側加 Search Index 方塊，
從 DB 畫一條虛線同步箭頭到 Search Index。

v7+ 是選配旁路，加一個方塊，不改動主線。



面試不是刷題



系統設計面試不是背標準答案。

它在看你能不能把需求、估算、架構、取捨講清楚。

先求有邏輯，再求有深度。



五步驟開場

問需求 → 估規模 → 畫 v1

找爆點 → 講取捨

每題都照這個順序走。



重點

不要跳步，跳步就容易亂。



先問需求

使用者是誰？核心功能是什麼？
讀多還是寫多？資料能不能慢一點同步？
哪個維度不能犧牲？



用戶是誰



核心功能



讀寫比例



延遲要求



一致性要求



失敗容忍

需求決定架構，不是技術名詞決定架構。



估算不用怕



DAU → 尖峰 QPS

請求壓力



每筆資料大小 → 儲存量

儲存壓力



每秒流量 → 頻寬壓力

頻寬壓力

只要估到量級，就足夠支撐架構討論。



不要先畫圖



一聽題目就畫微服務，通常是扣分。

先畫最小可行 v1，再根據爆點加能力區塊。

這就是本課 v1 → v7 的用法。



一句話講取捨

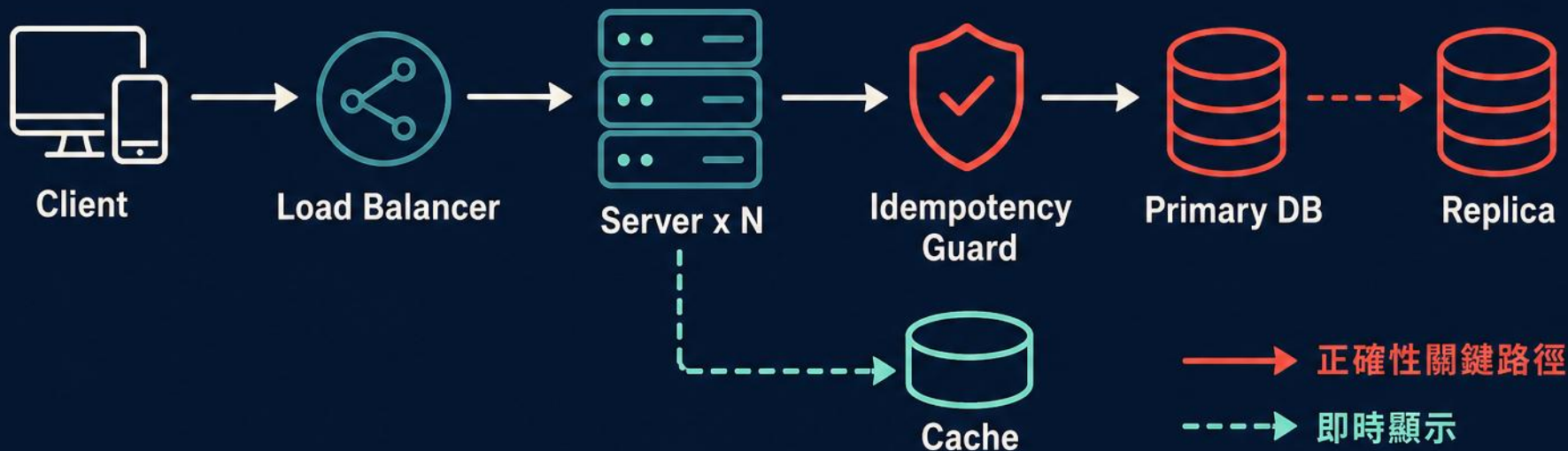
在 **X** 約束下，我選 **Y**。

代價是 **Z**，但它換來 **A**。

如果規模變大，我會升級成 **B**。

C	
A	
L	
Cost	

投票題示範



題目：百萬人同時投票，每人只能投一次。

寫入結果走 DB，重複投票用 Idempotency 擋。

即時票數可用 Cache，最終票數以 DB 為準。



面試官追問



DB 掛了怎麼辦？

備援



有人連點兩次怎麼辦？

冪等



為什麼不用 Kafka / Redis / Elasticsearch？

取捨

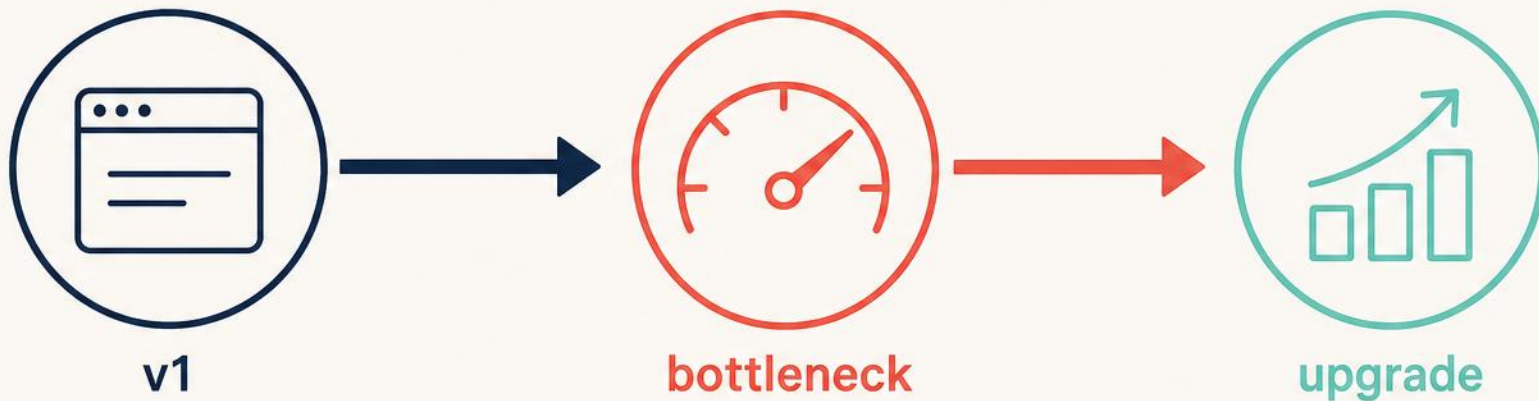
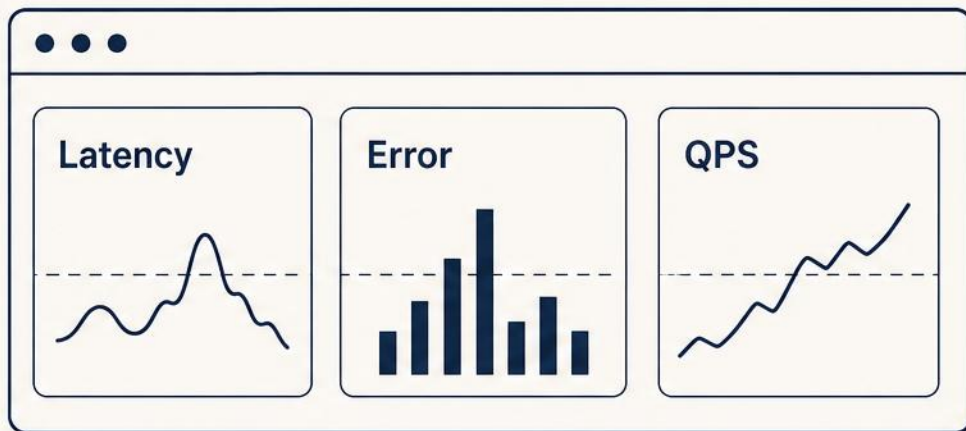


實務不是炫技

真實工作先量測瓶頸，不先堆技術。

先用簡單方案上線，再按痛點升級。

每個升級都要寫清楚代價。



最後小抄

先問需求，再估規模。

先畫 v1，再逐步升級。

每個技術都要講清楚取捨。

①  問需求

②  估規模

③  畫 v1

④  找爆點

⑤  講取捨

不要只說技術，要說理由。

100 萬+ 用戶



你已經走完了。來驗收。

從 10 個用戶一路撐到一百萬以上。

每次爆掉，你知道為何爆、用什麼解藥。

現在，換你面對一個全新的系統。



v1 到 v7

- v1 ● Client → Server → DB (最簡單的起點)
- v2 ● Load Balancer + Server × N (水平擴展)
- v3 ● Cache 層加入 (菜單不再重複查 DB)
- v4 ● Primary + Replica + Shard (資料層擴展)
- v5 ● 備援 + 冪等 (東西會壞，要能恢復)
- v6 ● CDN + Blob Storage (大型靜態資源)
- v7 ● Message Queue + Worker (非同步解耦)

能默畫這條線，就是這門課的終點。



四個維度，任何系統都能評

C

一致性

Consistency

所有人看到的資料一樣嗎？

A

可用性

Availability

系統掛掉了還能用嗎？

L

延遲

Latency

多快回應用戶的請求？

Cost

成本

Cost

要燒多少錢才能撐住？

沒有最好的答案，只有取捨。



面對新系統，先問這三步

① 痛點

這個系統在什麼情況下會爆？

② 解藥

有哪些技術能解這個痛？

③ 取捨

選這個技術，C/A/L/Cost 各犧牲什麼？

① 痛點：這個系統在什麼情況下會爆？

② 解藥：有哪些技術能解這個痛？

③ 取捨：選這個技術，C/A/L/Cost 各犧牲什麼？

這三步，是你看任何新技術文章的閱讀框架。



新題目：設計線上投票系統

新題目

設計線上投票系統

百萬用戶同時投票

每票只能投一次

結果必須準確

百萬人同時投票，每票只能投一次。

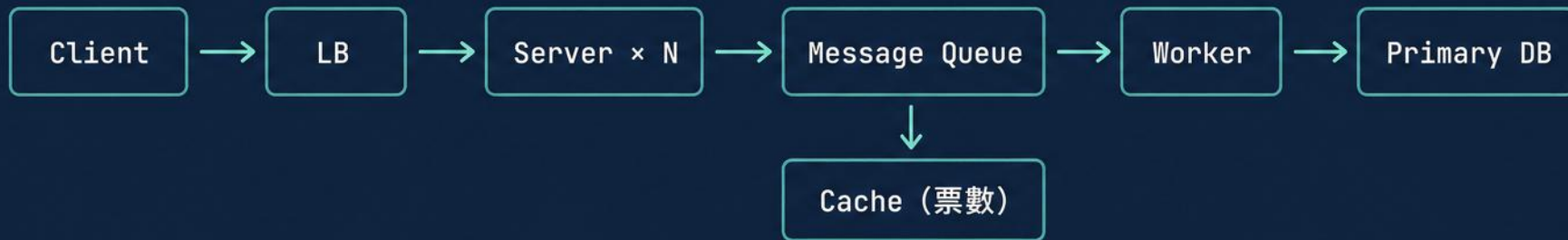
先問自己：C/A/L/Cost 哪個最重要？

投票防灌票 → 強一致性是第一優先。



選解藥畫圖

你的架構圖草稿



LB 分流 → DB 寫入投票 (Primary, 強一致)

Queue 削峰, Worker 慢慢寫入結果

Cache 存即時票數, DB 存最終正確票數



三件事，確認你學到了



我能默畫點餐 App 的 v1 到 v7，不看提示



我能選出至少 2 個技術，並說出各自的取捨



我看技術文章不再被名詞嚇到，會先問它解什麼痛

打勾打不完全也沒關係，知道哪裡還不熟，就是下一步的方向。



NEXT

想再深一點，往這些方向走

一致性模型

強一致 vs 最終一致，CAP 定理的直覺理解

可觀測性

日誌、指標、追蹤——系統如何知道自己的狀態

分散式系統進階

訊息系統深入、分散式交易、服務網格概念

這門課給你地圖，接下來你自己探索。

你已經不是對架構圖一無所知的初學者了。

